

Concurrent Programming in Dyalog

Load Code Samples

Download directly from the internet into the active workspace:

```
]get -u https://github.com/dyalog-  
training/2026-MA3  
Working on it...  
#.Performance #.Seats #.WordSearch
```

If you have downloaded the repository as a zip, unzip and import it:

```
]get -u "C:\Users\rpark\Downloads\2026-MA3-  
main\2026-MA3-main"  
#.Performance #.Seats #.WordSearch
```

Parallelisation Performance

```
Collatz 2 2e6  
chunks←10 Splits 2 2e6  
Best Collatz ``chunks  
Best [TSYNC Collatz& ``chunks  
Best Collatz IĪ chunks
```

One collatz step is encoded in `CollatzStep`. The function `Collatz` gives the number of steps and starting number of the longest sequence in range $m \dots n$.

For example, it takes 118 steps to go from 97 to 1:

```

Collatz 2 100
118 97
    i←0 ⋄ CollatzStep×{i+←1 ⋄ 1=α} 97 ⋄ ⍵←i
1
118

```

Use the `_Time` operator to measure run time of expressions.

```

Collatz _Time 2 2e6

chunks←10 Splits 2 2e6

{Best Collatz`ω}_Time chunks
{Best ⍵TSYNC Collatz&`ω}_Time chunks
{Best Collatz Iİ`ω}_Time chunks

```

1. Before running, guess an order 1 (fastest) to 4 (slowest) of the relative performance of each version
2. Try to estimate run times of different versions based on number of processors in your machine and the average run time of a single chunk.
3. Try using fewer chunks and more chunks than the number of processors. What happens to performance?
4. Why do these expressions have these performance characteristics?

APL Threads

Run `function` in a new thread with `spawn function&`. The result is the thread ID. Get array of results from array of thread IDs with `⍵TSYNC`. This blocks until all threads have completed. `⍵TSYNC` errors with `DOMAIN ERROR` if called on a completed thread. It should be used inline with `spawn` calls.

```

    tid ← function&arg
    r ← TSYNC tid
VALUE ERROR: No result was provided when the
context expected one
    r ← TSYNC tid
    ^

    TSYNC function&arg
result

```

Isolates

Copy isolates `CY 'isolate'` (must be in `#`) Run an expression in an isolate `function II`. The result is a **future**. Parallel-each `function II`. Returns an **array of futures**.

Futures may be restructured and passed around, only blocking when the value is required.

```

fut ← {_←DL 10 ◊ ω}II 5
r ← 10 10 ϕ fut
r ← 1 0 1 0 0 ◊ fut
r ← 3 ϕ fut

```

The Token Pool

Reserve a token range with `TALLOC`. Put tokens in the token pool with `TPUT`. Tokens are positive or negative real numbers. Tokens with the same magnitude (absolute value $|\omega|$) are said to be *of the same type*.

Get a token `TGET`. This blocks until the requested token is in the token pool.

For a positive token in the pool, only `TGET` of a positive token of the same type allows passage, and that token is removed from the pool.

For a negative token in the pool: - `TGET` for positive token of the same type allows passage without removing the token from the pool. - `TGET` for a negative token of the same type allows passage and removes the token from the pool.

This behaviour is summarised in the table below.

Pool	⌈TGET	Behaviour
+ve	+ve	Proceed; remove token from pool
+ve	-ve	Block
-ve	+ve	Proceed; -ve token remains in pool
-ve	-ve	Proceed; -ve token is removed from pool

⌈TGET and ⌈TPUT can also be used to store and retrieve values.

```
'Hello'⌈TPUT 3
⌈←⌈TGET 3    ⌘ The result is enclosed
Hello
```

Handling Shared State

Isolates provide a simple interface to deal with in-progress results. Obtain the result of a completed future, else a fallback value.

```
my_future ← DoThing II arg
default ← 'NOT READY YET'
r ← default #.isolate.Values 'my_future'
```

In APL threads, we must be careful when dealing with global values.

```
▽ {r}←Increment n;i;tmp
  :For i :In n
    tmp←G      ⌘ Take a snapshot. A thread may switch here.
    G←tmp+1    ⌘ Update state. Our snapshot might be out of date.
  :EndFor
  r←0
▽
```

Increment by 200,000 four times. The result should be 800,000.

```
G←0 ⌘ Increment``4p2e5 ⌘ G
800000
G←0 ⌘ ⌈TSYNC Increment&``4p2e5 ⌘ G
771436
```

Single lines of APL will not switch threads. Thread switch points include: – between lines of code – on entry to dfn or dop – awaiting completion of an external operation

Read more details at <https://docs.dyalog.com/20.0/programming-reference-guide/threads/thread-switching/>

Threads can "block" other threads by holding tokens. There are two token-holding mechanisms.

The `:Hold` control structure uses an arbitrary character scalar or vector as an application-wide token. It is released at `:EndHold` or if the thread terminates for any reason. Trailing spaces in `:Hold` tokens are ignored.

`⌈TPUT/⌈TGET` are more fine-grained, but should be used with care. See documentation, including examples at <https://docs.dyalog.com/20.0/programming-reference-guide/threads/multithreading-overview/>.